

Università degli Studi di Roma "Tor Vergata"
Laurea in Informatica

Sistemi Operativi e Reti
(modulo Reti)
a.a. 2025/2026

Livello di applicazione (parte 3)

dr. Manuel Fiorelli

manuel.fiorelli@uniroma2.it

<https://art.uniroma2.it/fiorelli>

Basate sulle slide del libro di testo:

https://gaia.cs.umass.edu/kurose_ross/ppt.php

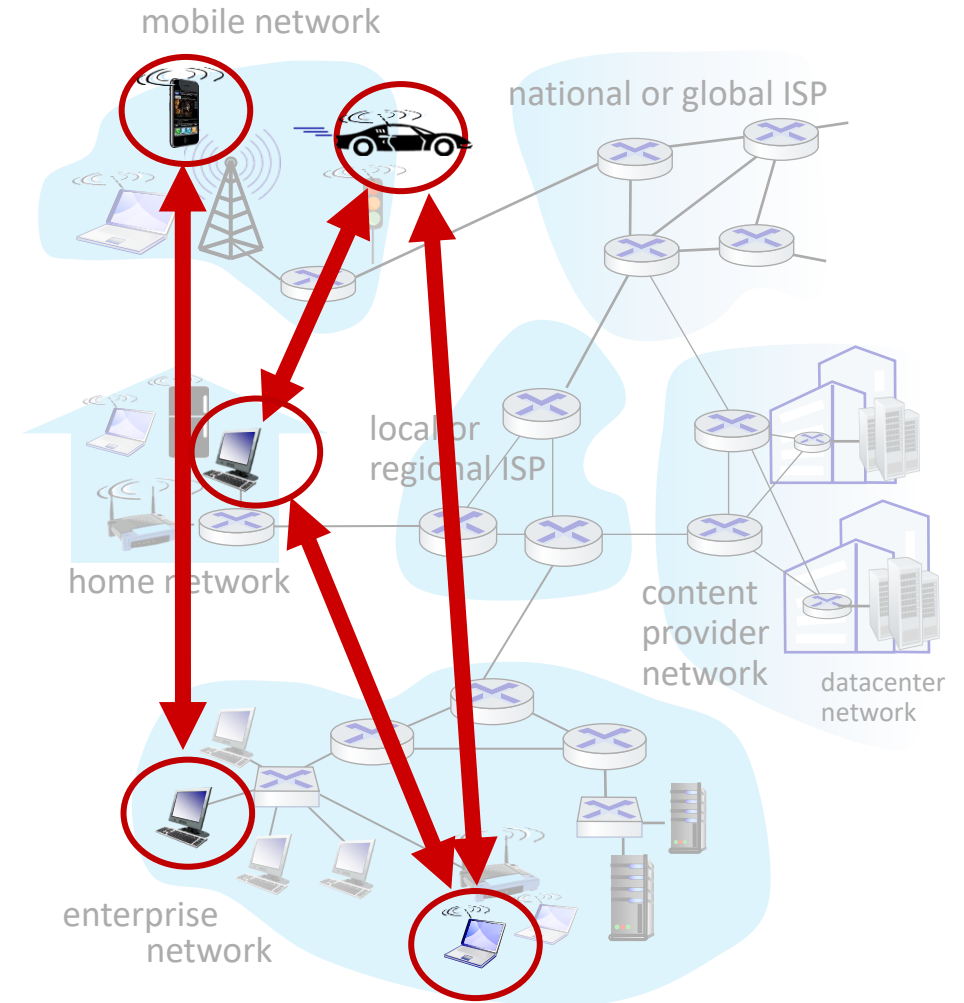
Application Layer: Overview

- Principi delle applicazioni di rete
- Web e HTTP
- E-mail, SMTP, IMAP
- DNS: il servizio di directory di Internet
- Applicazioni P2P
- Streaming video e reti di distribuzione di contenuti
- Programmazione delle socket programming con UDP e TCP



Architettura Peer-to-peer (P2P)

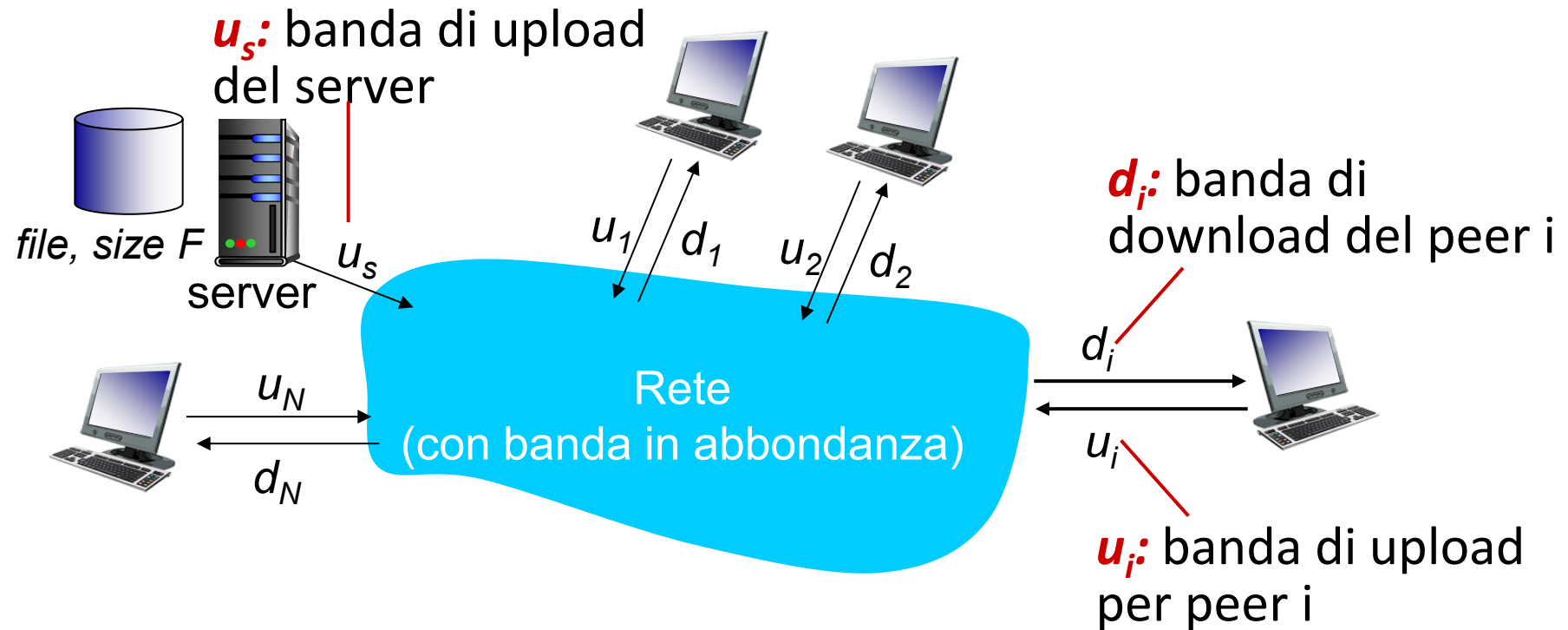
- *nessun* server sempre attivo
- sistemi periferici arbitrari comunicano direttamente
- i peer richiedono un servizio ad altri peer e forniscono un servizio in cambio ad altri peer
 - *scalabilità intrinseca - nuovi peer portano nuova capacità di servizio e nuove richieste di servizio*
- I peer sono connessi a intermittenza e cambiano indirizzo IP
 - gestione complessa
- Esempi: P2P file sharing (BitTorrent), streaming (KanKan), VoIP (Skype)



Distribuzione di file: client-server vs P2P

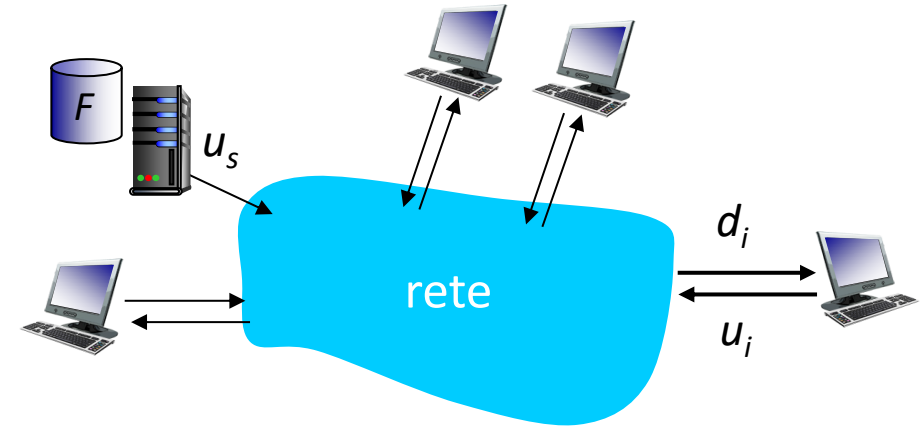
D: quanto tempo per distribuire un file (di dimensione F) da un server a N peer?

- la capacità di upload/download dei peer è una risorsa limitata



File distribution time: client-server

- **trasmissione via server:** deve inviare (caricare) in sequenza N copie di file:
 - tempo per inviare una copia: F/u_s
 - tempo per inviare N copie: NF/u_s
- **client:** ogni client deve scaricare una copia del file
 - d_{min} = banda di download più bassa
 - tempo di download per il client con banda minima è almeno: F/d_{min}



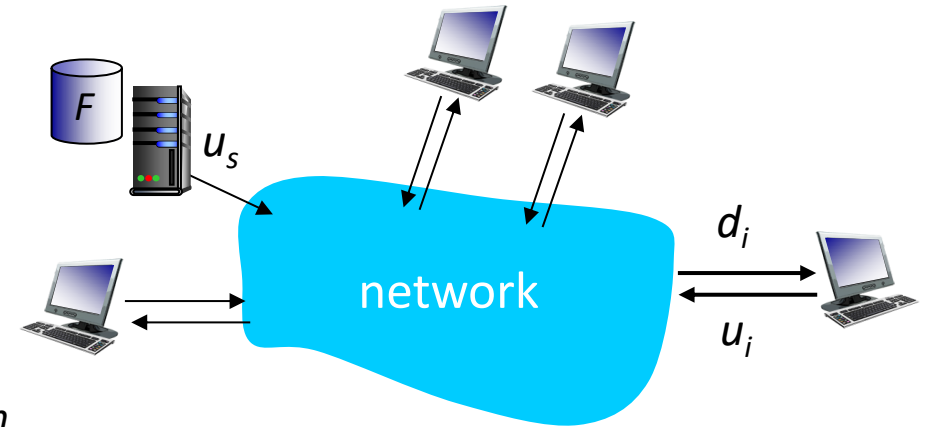
*Tempo per distribuire F
a N client usando
l'approccio client-
server*

$$D_{c-s} \geq \max\{NF/u_s, F/d_{min}\}$$

aumenta linearmente in N

Distribuzione di file: P2P

- *trasmissione via server*: deve trasmettere almeno una copia del file:
 - tempo per inviare una copia: F/u_s
- *client*: ogni client deve scaricare una copia del file
 - Tempo per il client più lento, almeno F/d_{min}
- I *client*: come aggregato devono scaricare NF bit
 - capacità totale di upload (che limita la massima velocità di download) è $u_s + \sum u_i$



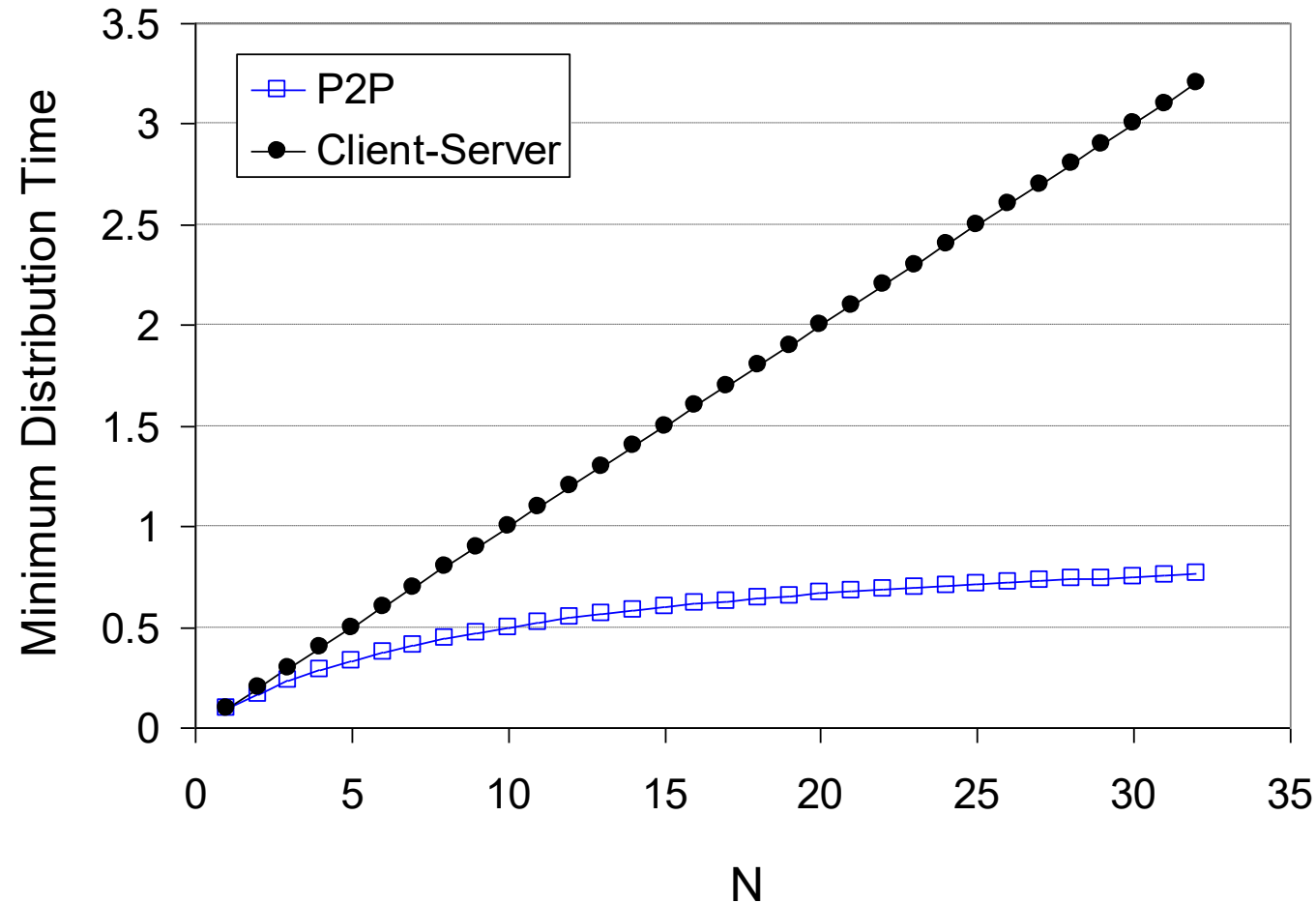
Tempo per distribuire F
a N client usando
l'approccio P2P

$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

aumenta linearmente in N ...
... ma anche questo, dato che ogni peer porta con sé la capacità di servizio

Client-server vs. P2P: example

banda di upload del client = u , $F/u = 1$ ora, $u_s = 10u$, $d_{min} \geq u_s$

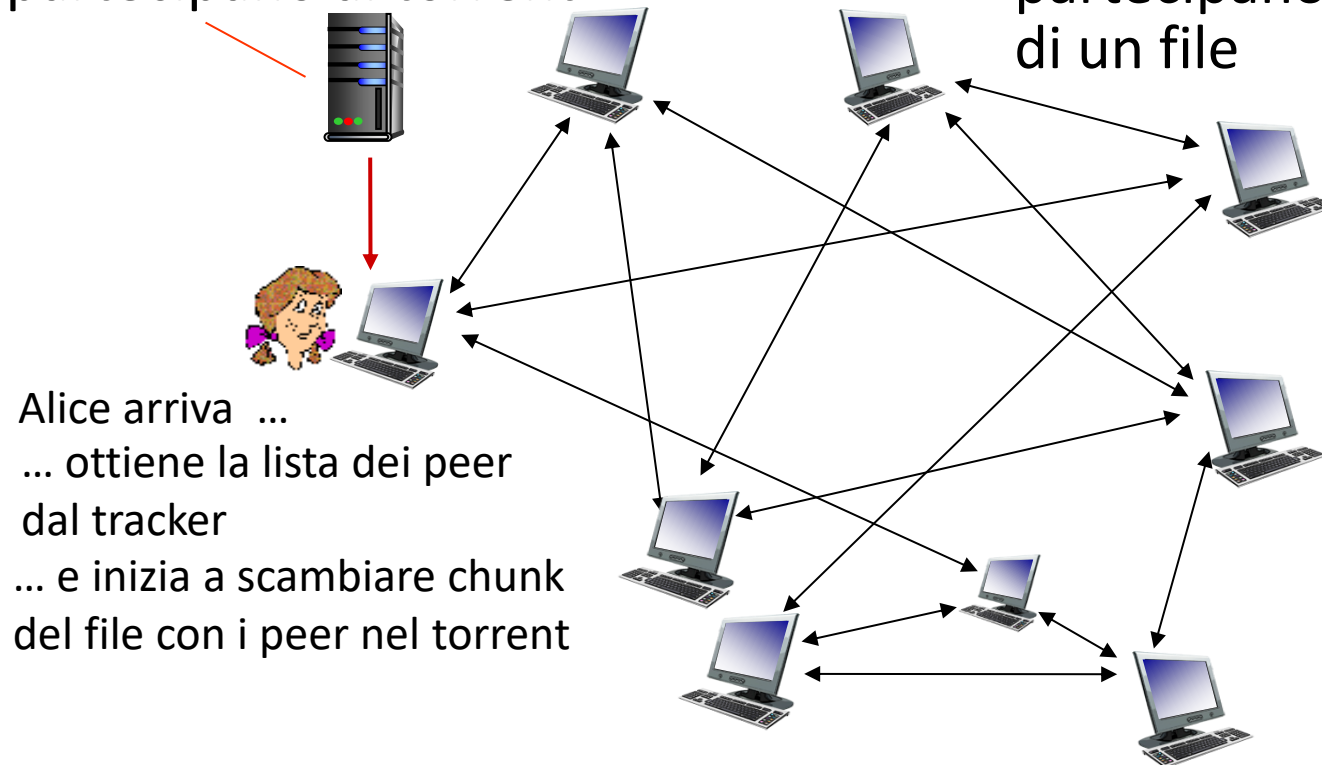


Distribuzione di file P2P: BitTorrent

- file diviso in chunk (parti), in genere di 256 kB
- i peer nel torrent inviano/ricevono chunk del file

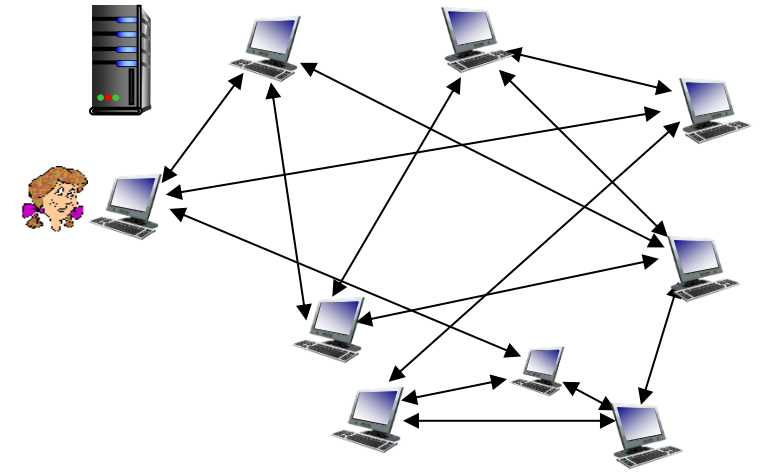
tracker: tiene traccia dei peer che partecipano al torrent

torrent: gruppo di peer che partecipano alla distribuzione di un file



Distribuzione di file P2P: BitTorrent

- Un peer che entra a far parte del torrent:
 - non ha chunk del file, ma li accumulerà nel tempo da altri peer
 - si registra con un tracker, ottenendo la lista di un sottoinsieme dei peer nel torrent (es. 50), stabilisce una connessione con un sottoinsieme di questi, che sono detti peer "vicini" ("neighbors")
 - informa periodicamente il tracker che è ancora nel torrent
- mentre scarica chunk, un peer invia i chunk già in suo possesso agli altri peer
- un peer può cambiare i peer con cui scambia i chunk
- i peer possono andare e venire
- una volta che un peer ha acquisito l'intero file, può (egoisticamente) lasciare il torrent oppure può (altruisticamente) rimanere nel torrent (come *seeder*)



BitTorrent: richiesta e invio di chunk di file

Richiesta di chunk:

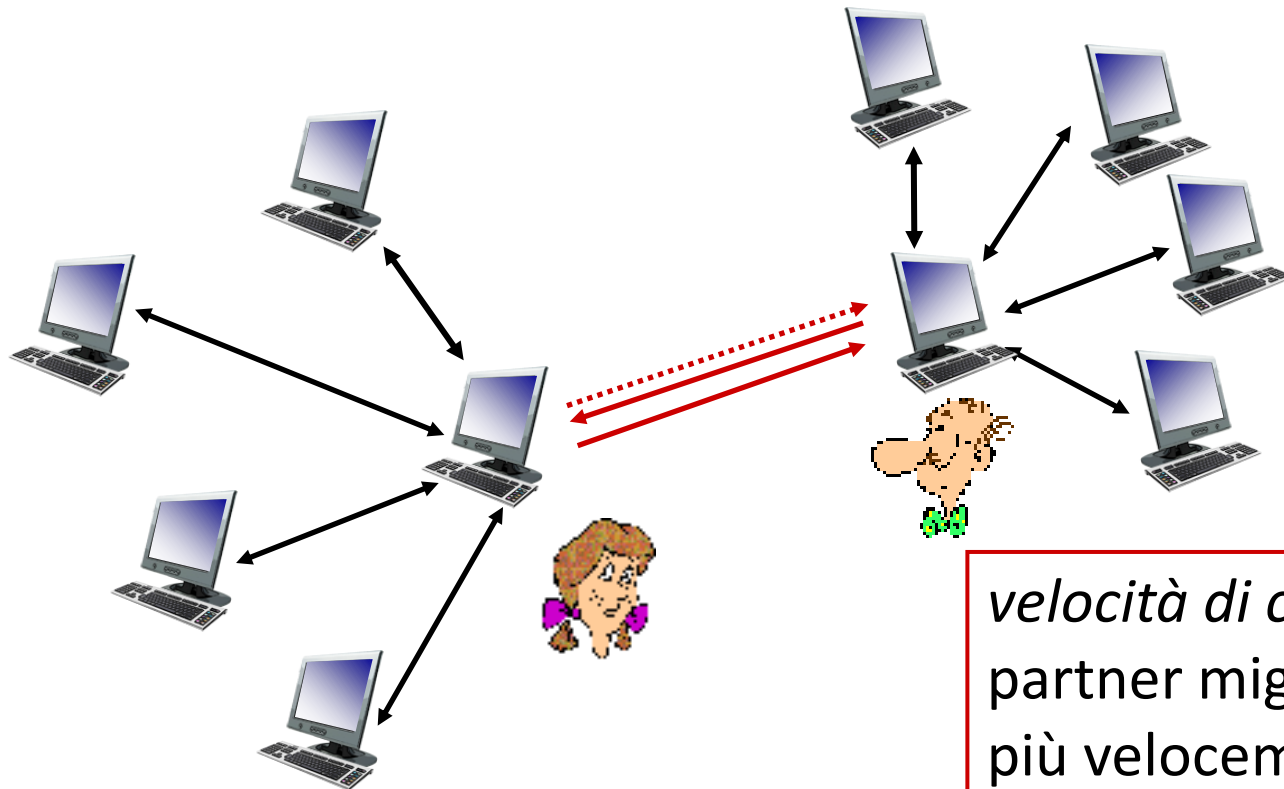
- in ogni momento, peer diversi hanno sottoinsiemi diversi di chunk
- periodicamente, Alice chiede ai peer vicini l'elenco dei chunk in loro possesso
- Alice richiede ai peer i chunk mancanti, adottando la strategia del **rarest first** ("prima i più rari"): uniformando la distribuzione dei chunk, migliora la disponibilità globale e aumenta le possibilità di scambio (maggiore throughput)
- Un peer appena entrato può chiedere un blocco in modo casuale (perché vuole avere il prima possibile un blocco da condividere); mentre, quando sta per completare il file, può adottare la strategia end game e richiedere lo stesso blocco a più peer simultaneamente (cancellando le richieste pendenti appena appena riceve un blocco)

Invio di chunk: tit-for-tat ("pan per focaccia")

- Alice invia i chunk ai quattro peer vicini che attualmente le inviano i chunk *alla velocità più alta*
 - altri peer sono detti choked ("soffocati" o "limitati") (non ricevono chunk da Alice)
 - rivaluta i primi 4 posti ogni 10 secondi
- ogni 30 secondi: seleziona in modo casuale un vicino, inizia a inviare chunk
 - questo peer è detto "optimistically unchoked" ("non limitato/soffocato in maniera ottimistica")
 - il nuovo peer scelto può entrare nella top 4

BitTorrent: tit-for-tat

- (1) Alice sceglie Bob come “optimistically unchoked”
- (2) Alice diventa uno dei primi quattro fornitori di Bob; Bob ricambia
- (3) Bob diventa uno dei primi quattro fornitori di Alice.



velocità di caricamento più elevata: trovare partner migliori per gli scambi, ottenere file più velocemente!

Livello di applicazione: panoramica

- Principi delle applicazioni di rete
- Web e HTTP
- E-mail, SMTP, IMAP
- DNS: il servizio di directory di Internet
- Applicazioni P2P
- Streaming video e reti di distribuzione di contenuti
- Programmazione delle socket programming con UDP e TCP



Streaming video e CDN: contesto

- traffico video in streaming :
grande consumatore di larghezza
di banda Internet
 - Netflix, YouTube, Amazon Prime: 80% del
traffico ISP residenziale (2020)
- *sfida*: scala - come raggiungere
~1B di utenti?
- *sfida*: eterogeneità
 - utenti diversi hanno capacità diverse (ad esempio, cablati
o mobili; ricchi di larghezza di banda o poveri di larghezza
di banda)
- *soluzione*: infrastruttura distribuita a livello di
applicazione

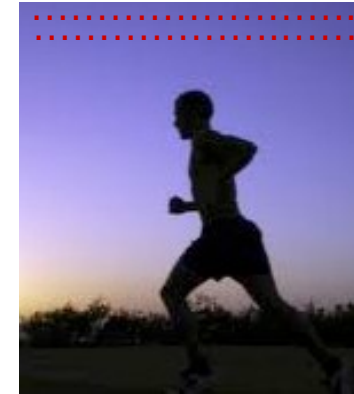


Contenuti multimediali: video

- video: sequenza di immagini visualizzate a tasso costante (*frame rate*)
 - Esempio: 24 immagini al secondo
- immagine digitale: un array di pixel
 - ogni pixel rappresentato da bit
- codifica: utilizzare la ridondanza *all'interno* e *tra* le immagini per ridurre il numero di bit utilizzati per la codifica dell'immagine
 - spaziale (all'interno di una data immagine)
 - temporale (da un'immagine all'altra)

esempio di codifica spaziale:

invece di inviare N valori dello stesso colore (tutti viola), inviare solo due valori: valore del colore (viola) e numero di valori ripetuti (N)



frame *i*

esempio di codifica temporale:

invece di inviare il frame completo a *i+1*, invia solo le differenze dal frame *i*



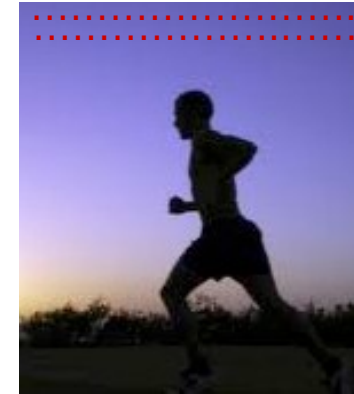
frame *i+1*

Multimedia: video

- **CBR: (constant bit rate):** bit rate costante
- **VBR: (variable bit rate):** bit rate cambia con la quantità di ridondanza spaziale e temporale
- **esempio:**
 - MPEG-1 (CD-ROM) 1.5 Mbps
 - MPEG-2 (DVD) 3-6 Mbps
 - MPEG-4 (spesso usato in Internet, 64kbps – 12 Mbps)

esempio di codifica spaziale:

invece di inviare N valori dello stesso colore (tutti viola), inviare solo due valori: valore del colore (viola) e numero di valori ripetuti (N)



frame *i*

esempio di codifica temporale:

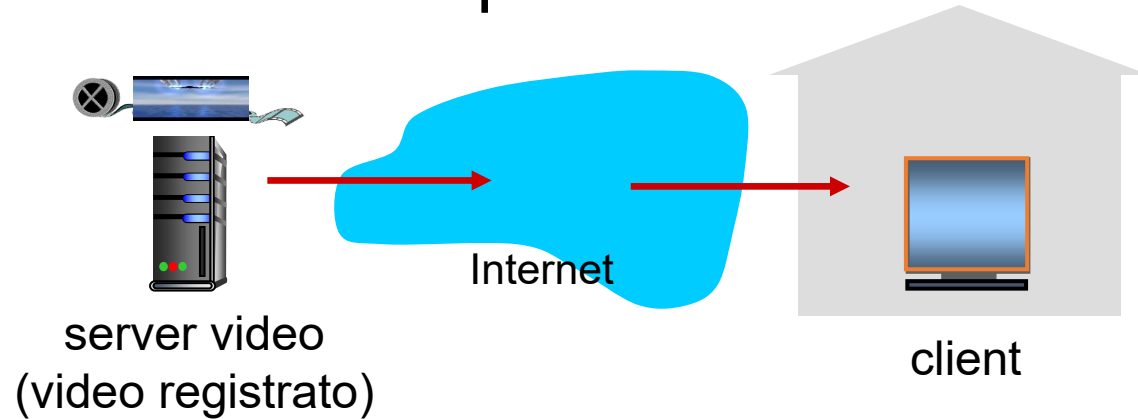
invece di inviare il frame completo a *i+1*, invia solo le differenze dal frame *i*



frame *i+1*

Streaming video di contenuti registrati

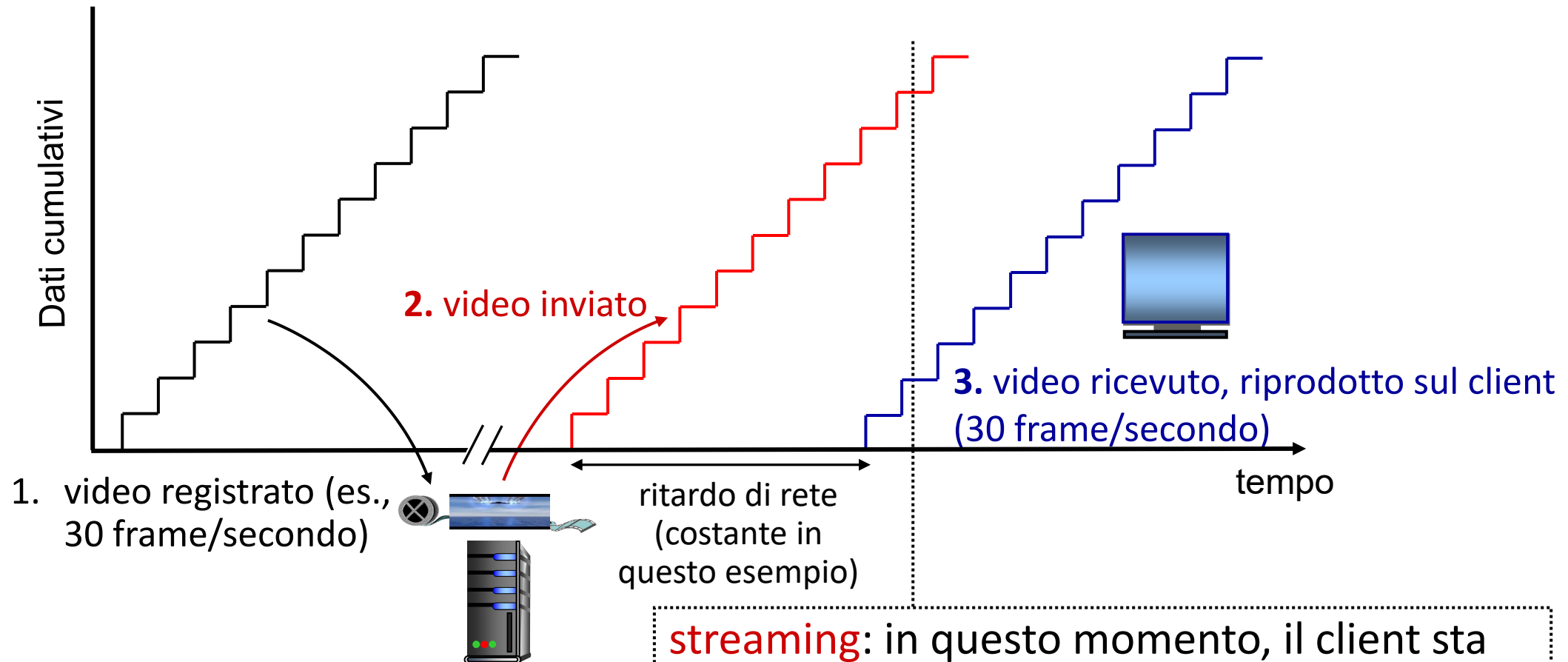
scenario semplice



Sfide principali:

- la larghezza di banda da server a client varia nel tempo, con il variare dei livelli di congestione della rete (rete residenziale, rete di accesso, nucleo della rete, server video)
- la perdita di pacchetti e i ritardi dovuti alla congestione ritardano la riproduzione o comportano una scarsa qualità video.

Streaming video di contenuti registrati

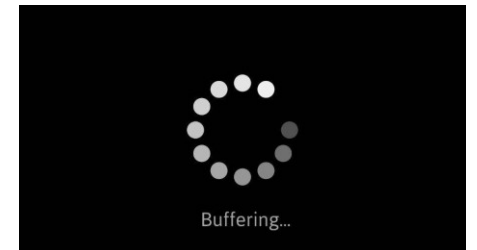


Limita il ritardo di riproduzione, evita spreco di banda se il client interrompe riproduzione

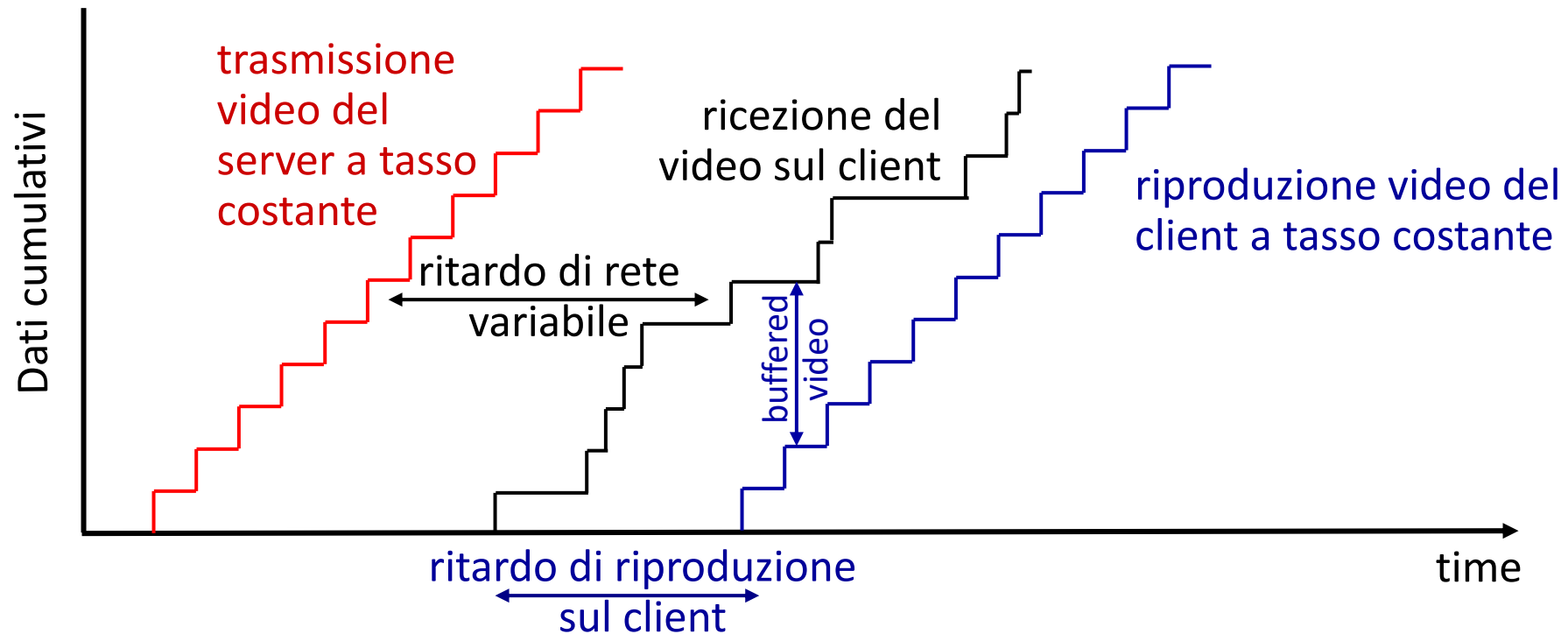
streaming: in questo momento, il client sta riproducendo la parte iniziale del video, mentre il server sta ancora inviando la parte successiva del video

Streaming video di contenuti registrati

- **vincolo di riproduzione continua**: quando la riproduzione inizia, dovrebbe procedere secondo i tempi di registrazione originali
 - ... ma **i ritardi di rete sono variabili** (jitter), quindi avrà bisogno di un buffer **lato client** per soddisfare i vincoli di riproduzione continua
- altre sfide:
 - interattività del client: pausa, avanzamento veloce, riavvolgimento, salti attraverso il video
 - i pacchetti video possono essere persi, ritrasmessi



Streaming video di contenuti registrati



- ***buffering lato client e ritardo di riproduzione:*** compensare il ritardo aggiunto dalla rete, il jitter (variazione) del ritardo

Streaming video di contenuti registrati

■ Streaming UDP:

- Il server invia pacchetti video in modo da eguagliare il bit rate del video stesso, traendo vantaggio dall'assenza di controllo della congestione (es. bit rate 2 Mbps e pacchetti da 8000 bit → il server invia un pacchetto ogni 4 ms)
- Buffer lato client equivalente a pochi secondi di video
- Connessione di controllo separata attraverso il quale il client può inviare comandi, quali pausa, salto, etc.
- Incapace di rispettare il vincolo di riproduzione continua, se la banda disponibile scende periodicamente sotto al bit rate del video (che è il tasso di consumo)

■ Streaming HTTP

- Il server trasmette alla massima velocità consentita (es. dal controllo della congestione): se la velocità di ricezione è maggiore del bit rate del video, il buffer del client continua a crescere anche durante la riproduzione (*prefetching*) fino al riempimento, dopodiché il controllo del flusso limiterà il tasso di trasmissione al tasso di consumo del client
- Può assorbire fluttuazioni del throughput, finché il throughput medio sia maggiore del bit rate
- Riga di intestazione *Range* nelle richieste per saltare

■ streaming dinamico adattativo su HTTP

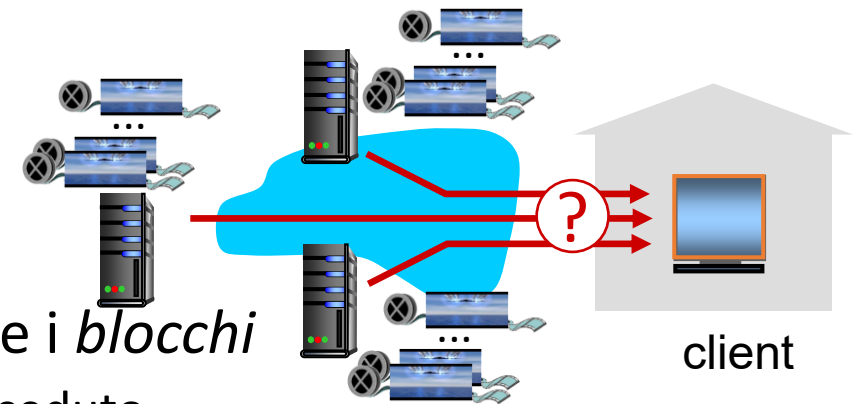
- Consente di scegliere tra versioni con livelli di qualità differenti anche durante la riproduzione

Streaming multimediale: DASH

Dynamic, Adaptive
Streaming over HTTP

server:

- il video viene codificato in più *versioni* con bit rate differenti (quindi, qualità differenti)
- ogni versione è suddivisa in *blocchi* (segmenti)
- *manifest file (file manifesto)*: file XML che descrive le versioni e come richiedere i *blocchi*
 - un *blocco* può avere una propria URL oppure essere acceduto attraverso una *HTTP range request* per la stessa URL associata alla versione

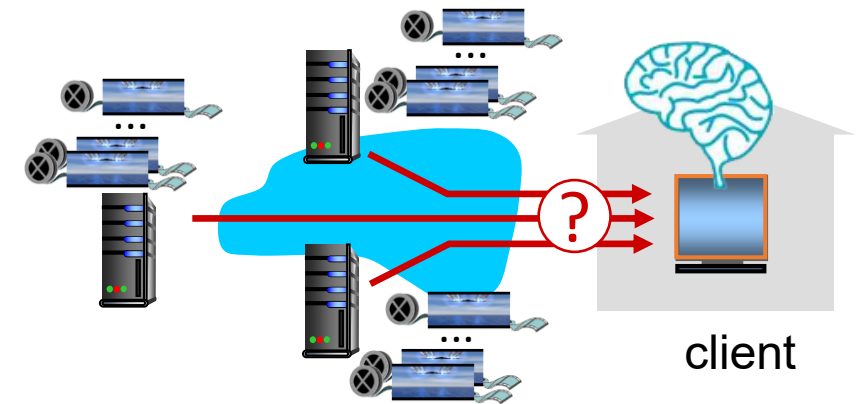


client:

- stima periodicamente la banda da server a client
- consultando il manifesto, richiede un *blocco* alla volta
 - sceglie versione con il bit rate più alto sostenibile data la larghezza di banda corrente
 - può scegliere versioni con bit rate differenti in momenti diversi (a seconda della larghezza di banda disponibile in quel momento), e da server diversi

Streaming multimediale: DASH

- “*intelligenza*” sul client: il client determina
 - *quando* richiedere un *blocco* (in modo che non si verifichi la starvation del buffer o l'overflow)
 - *che bit rate richiedere* (qualità più alta quando c'è più banda disponibile)
 - *dove* richiedere il *blocco* (può richiedere dal server che è "vicino" al client o ha banda larga)



Lato server: è sufficiente un server Web in grado di servire contenuti statici (file manifesto e blocchi)

Streaming video = codifica + DASH + buffering di riproduzione

Reti per la distribuzione di contenuti - Content distribution networks (CDNs)

sfida: come trasmettere contenuti in streaming (selezionati tra milioni di video) a centinaia di migliaia di utenti simultanei?

- *opzione 1*: unico, enorme data center
 - singolo punto di rottura (single point of failure)
 - punto di congestione della rete
 - percorso lungo (e possibilmente congestionato) verso i clienti lontani

.... molto semplicemente: questa soluzione *non è scalabile*

Reti per la distribuzione di contenuti - Content distribution networks (CDNs)

sfida: come trasmettere contenuti in streaming (selezionati tra milioni di video) a centinaia di migliaia di utenti simultanei?

■ *opzione 2:* memorizzare/servire più copie di video in più siti geograficamente distribuiti (*CDN*)

- *enter deep (entrare in profondità):* installare server della CDN in profondità dentro molte reti di accesso

- essere vicini agli utenti -> minore ritardi e maggior throughput, ma maggiore complessità di gestione e manutenzione
 - Akamai: 240,000 server dispiegati in > 120 paesi (2015)

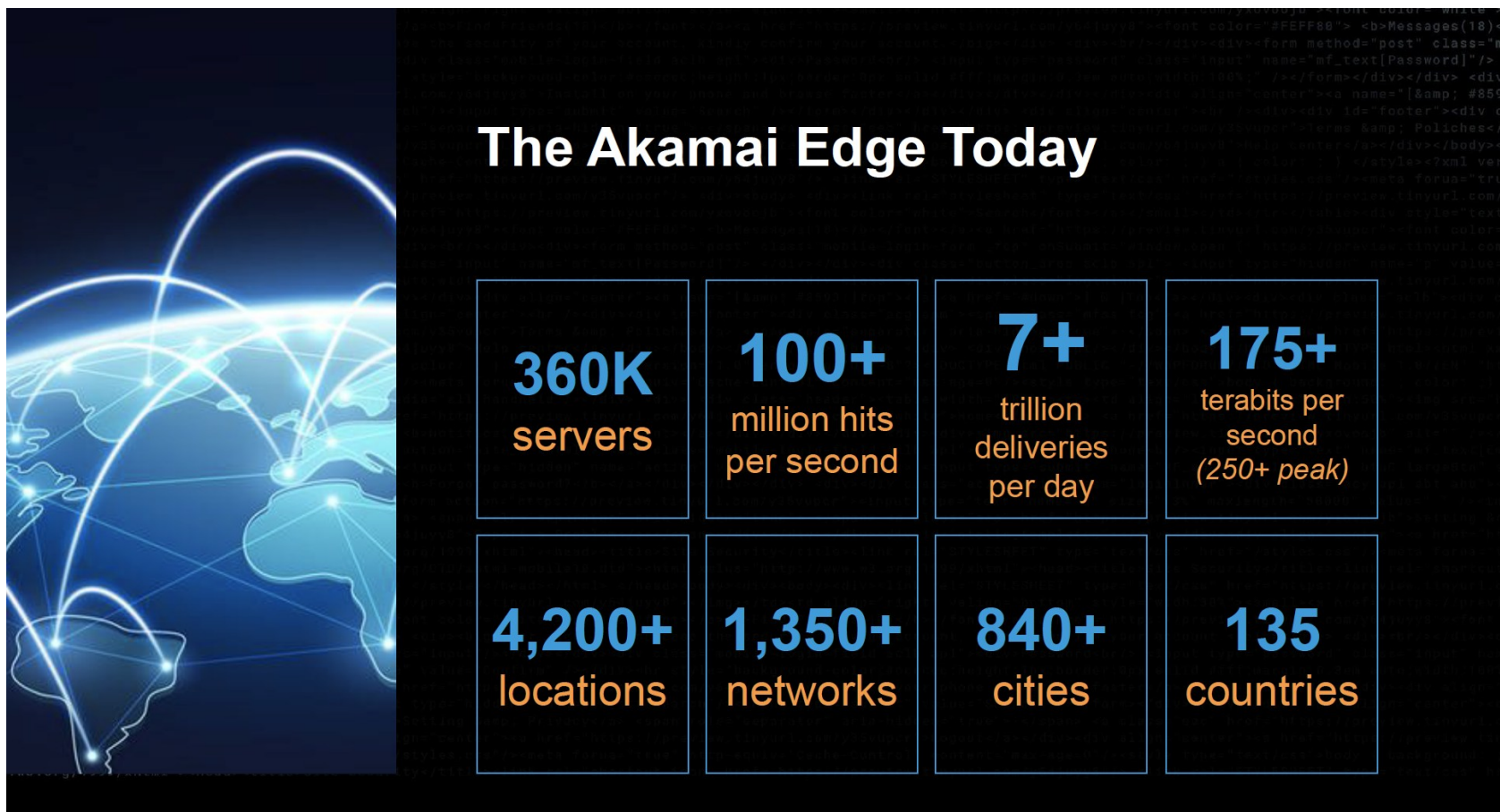


- *bring home (portare a casa):* pochi grandi cluster (decine, per esempio) in IXP vicino alle reti di accesso

- usato da Limelight



Akamai oggi:

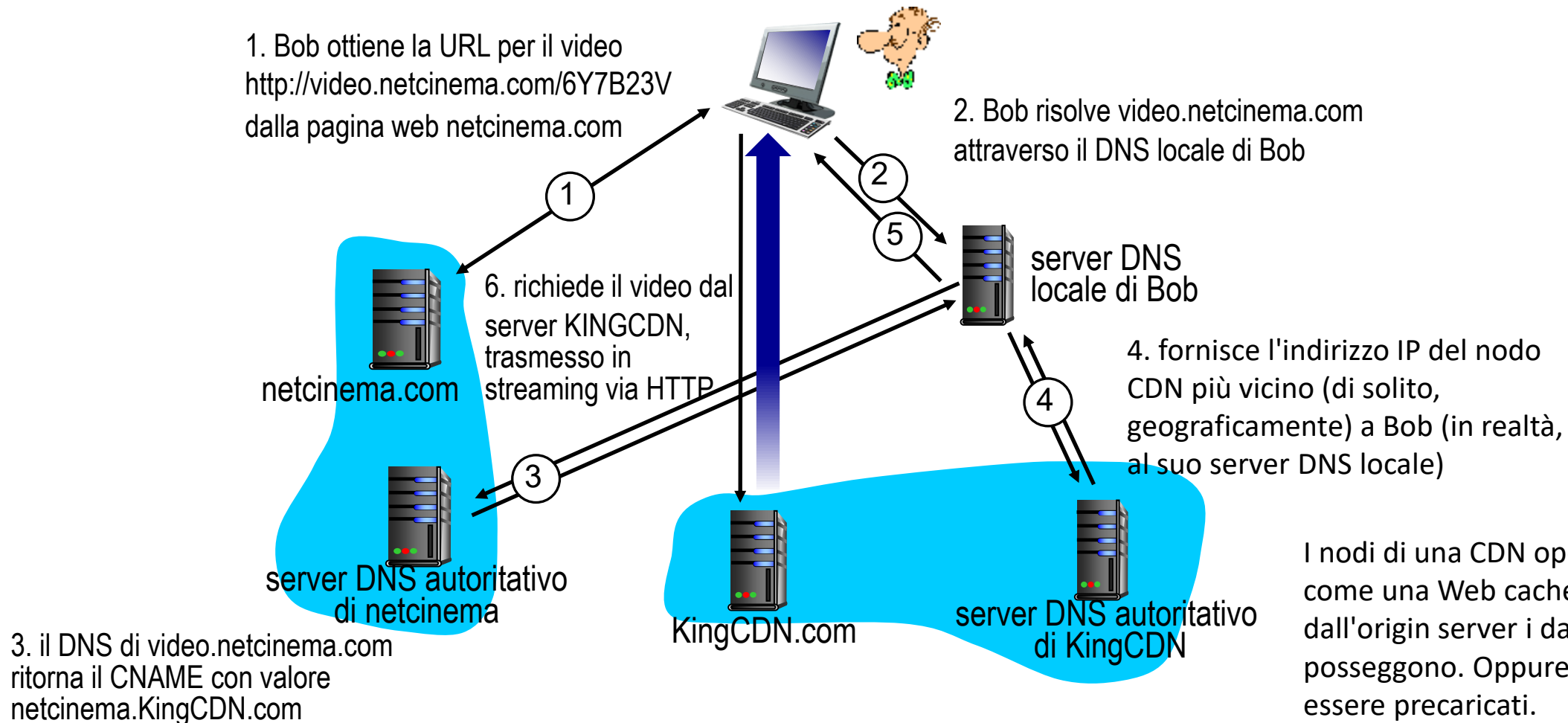


Fonte: <https://networkingchannel.eu/living-on-the-edge-for-a-quarter-century-an-akamai-retrospective-downloads/>

Accesso ai contenuti CDN: uno sguardo da vicino

Bob (client) richiede il video `http://video.netcinema.com/6Y7B23V`

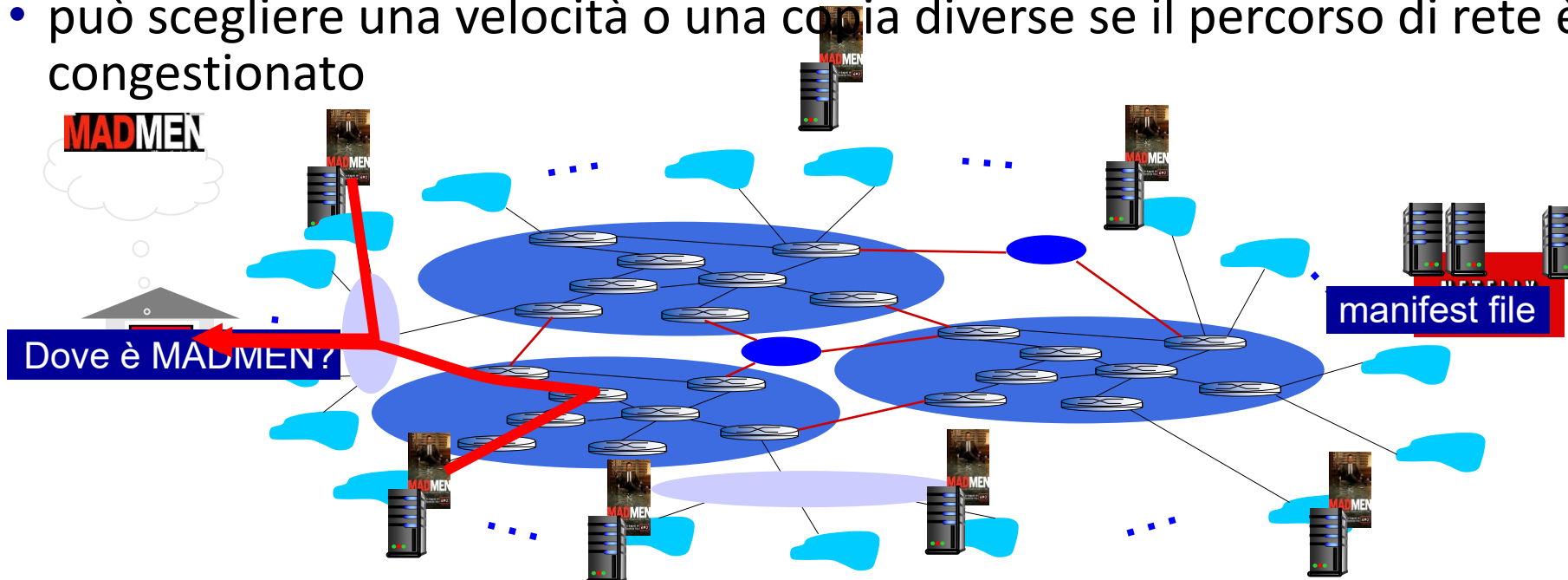
- video memorizzato sulla CDN a `http://netcinema.KingCDN.com/6Y7B23V`



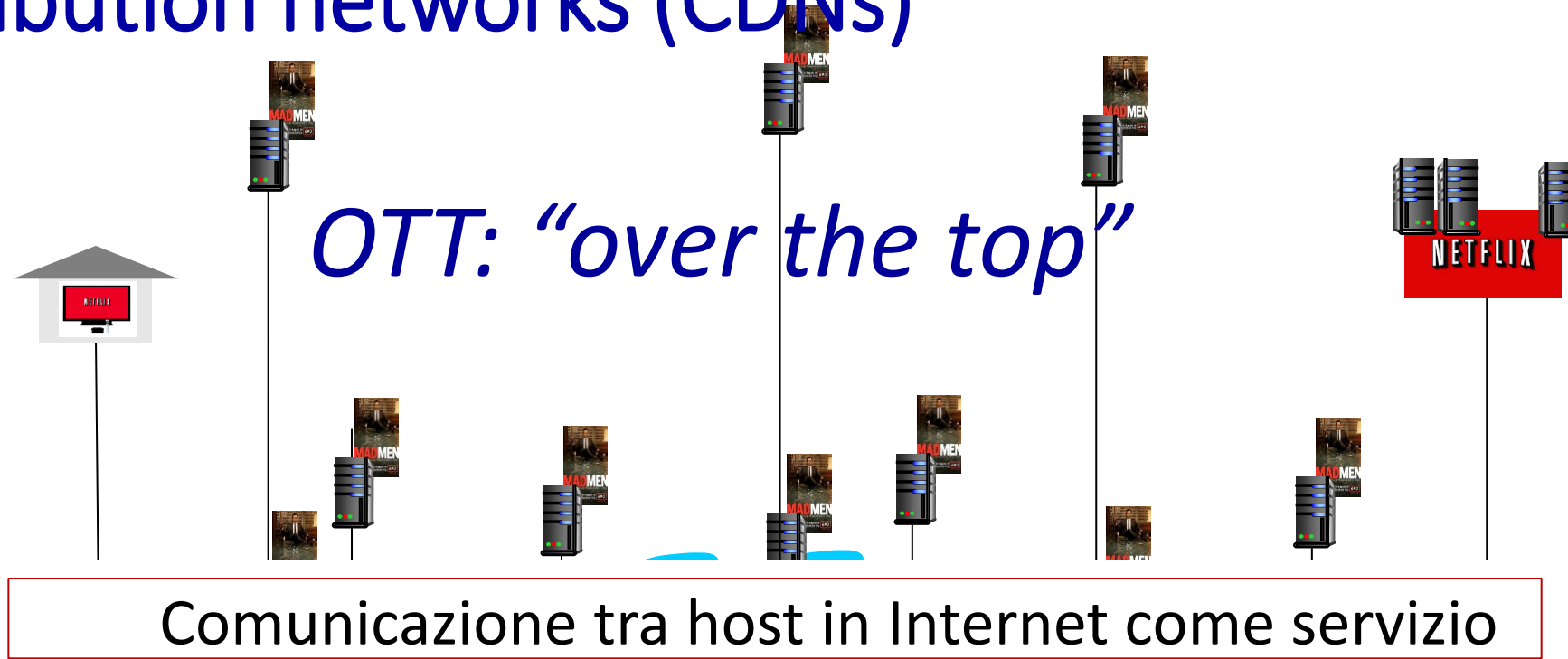
I nodi di una CDN operano solitamente come una Web cache recuperando dall'origin server i dati che non posseggono. Oppure, i dati possono essere precaricati.

Come funziona Netflix?

- Netflix: memorizza copie dei contenuti (ad esempio, MADMEN) sui nodi (in tutto il mondo) della sua CDN OpenConnect
- l'abbonato richiede il contenuto, il fornitore di servizi restituisce il manifesto
 - utilizzando il *manifest*, il client recupera i contenuti alla massima velocità supportabile
 - può scegliere una velocità o una copia diverse se il percorso di rete è congestionato



Reti per la distribuzione di contenuti - Content distribution networks (CDNs)



Sfide OTT: affrontare un Internet congestionato dal "bordo"

- quale contenuto inserire in quale nodo CDN?
- da quale nodo CDN recuperare i contenuti? A quale velocità?